Teacher-Guided QR-DQN: Distillation, Preferences, and Behaviour Cloning for Sample-Efficient Sequential Decision Making

Doruk Akay
Bachelor Thesis – Interim Report

June 29, 2025

Abstract

This thesis presents a teacher-guided reinforcement learning pipeline that significantly improves sample efficiency using three complementary techniques: policy distillation, preference-based reward relabelling, and behaviour cloning. A high-capacity Quantile-Regression DQN (QR-DQN) teacher is trained once on the CartPole environment. First, policy distillation transfers the teacher's softened action-value outputs to a compact student by minimizing the KL divergence. Second, short trajectory segments ranked by the teacher train a reward model that converts sparse environment feedback into a dense shaping signal. Finally, a lightweight behaviour-cloning term nudges the student toward transitions that have already received high shaped reward, without suppressing exploration. Combined, these three signals let the student reach strong performance after roughly one order of magnitude fewer interactions, substantially outperforming baselines trained from scratch. Ablation studies show that distillation and reward shaping provide most of the gain, while behaviour cloning mainly stabilises early learning. Because the teacher is trained offline and frozen, the method requires no human annotation and allows a small on-device agent to inherit the teacher's competence without deploying the large model itself.

1 Introduction

Deep reinforcement learning (RL) has reached superhuman scores in Atari [5], mastered the game of Go [6], and powered dexterous robot manipulation [7]. Despite these accomplishments, there exists a limitation: state-of-the-art agents frequently necessitate millions of interactions, which are impractical for genuine robots, impede industrial processes, or safety-sensitive domains where each erroneous action incurs a tangible consequence. Human involvement can alleviate this data scarcity by integrating domain expertise through demonstrations, corrective feedback, or reward optimisation. Regrettably, such guidance incurs significant costs, underscoring the necessity of algorithms that maximize utility from constrained supervisory cues.

One promising direction is distributional reinforcement learning (distributional RL), which enhances standard methods by predicting the entire spectrum of potential future rewards rather than merely a single expected value. This gives the agent a better understanding of uncertainty in its environment. A well-known version, quantile regression DQN (QR-DQN), learns multiple such reward estimates, known as quantiles, and computes a Q-value as their average [3]. These quantiles provide a more nuanced picture of the agent's confidence and help human supervisors identify risky or unfamiliar situations where intervention may be needed—much like uncertainty-driven exploration methods (e.g., the approach used in Rainbow DQN [8]) and PEBBLE's feedback scheduling [4] guide learning more efficiently. To optimise learning efficacy, various methodologies incorporate teacher guidance, a setup where a sophisticated, pre-trained agent (the "teacher") facilitates training a smaller or less experienced agent (the "student"). The key motivation is to compress the knowledge of a large, offline-trained teacher into a lightweight student that can keep learning online while meeting strict hardware and data-efficiency constraints. We realise this transfer through

three complementary signals, each aimed at a different bottleneck of online reinforcement learning. First, policy distillation allows the student to learn from the teacher's behaviour by mimicking its action choices, effectively transferring useful knowledge and reducing training time [9, 10]. Nevertheless, this approach alone may encounter challenges when the student encounters situations that the teacher has not encountered during their training.

Learning from preferences effectively addresses sparse or poorly defined rewards by soliciting a supervisor's selection of the superior trajectory snippet. This approach facilitates the fitting of a dense reward model [1], [11]. However, preference-based rewards can still be noisy, because the ranking signal—whether supplied by humans or, as in our case, a teacher model—may include inconsistencies and biases. This noise can lead the agent to acquire misleading reward signals and explore suboptimal behaviours unless appropriate constraints are implemented [1], [11].

Behaviour Cloning (BC) enhances reinforcement updates with a supervised loss that replicates actions observed in high-return trajectories. Traditional BC approaches rely on a separate expert to provide these trajectories, either through demonstrations [12] or offline data sets [13]. In this thesis, we adopt a self-imitation variant: the student stores its own transitions whose shaped reward exceeds a threshold τ in a buffer and subsequently maximizes its likelihood using a cross-entropy loss. This self-generated signal improves credit assignment without any additional teacher queries. When combined with policy distillation—which provides a robust initialization—and preference-based reward relabelling—which densifies sparse feedback—the resulting three signals act on distinct failure modes and collaborate to accelerate learning.

The literature has so far explored these ingredients mostly in isolation: distillation studies assume a fixed reward, preference papers rely on scalar critics, and BC is frequently treated as a pre-training add-on. What remains unclear is how the three mechanisms are connected and interact simultaneously to a distributional backbone. Do they sum up, interfere with, or even amplify one another? Guided by these considerations, the present study asks three concrete questions.

How much can teacher guidance improve the learning speed and final returns of a compact QR-DQN student relative to a scratch-trained baseline?

RQa — Policy distillation. How does training the student to imitate the action preferences of a well-performing teacher impact early learning and final returns?

RQb — Preference-based reward relabelling. How does replacing sparse environment rewards with a dense signal learned from pairwise trajectory rankings affect exploration efficiency and cumulative return?

RQc — Behaviour cloning of high-value transitions. In what way does adding a cross-entropy loss on teacher-approved transitions stabilize updates and contribute to higher or more reliable final returns?

RQd — Combined teacher-guided learning. What are the cumulative and interactive effects of combining all three guidance mechanisms—distillation, preference-based rewards, and behaviour cloning—on the student's performance?

This thesis proceeds by introducing relevant literature, our teacher guided QR-DQN framework, describing the experimental setup, analysing results across multiple guidance strategies and concluding with insights and directions for future research.

2 Related Work

Most value-based deep-RL agents predict a single scalar return for each state–action pair. While simple, that choice magnifies over-estimation bias and hides how outcomes vary. Distributional methods such as QR-DQN address both issues by learning the full return distribution, which provides smoother gradients and noticeably more stable updates on small networks. For these reasons we adopt QR-DQN as the critic in our teacher–student pipeline and use the mean of its quantiles as the value estimate supplied to the policy.

2.1 Deep Q-Networks: scalar value approximation

Bellman background. In reinforcement learning a task is formalised as a discounted Markov decision process (MDP)

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, r, \gamma \rangle.$$

Here $\mathcal{S}$ is the set of all possible states of the environment, $\mathcal{A}$ is the set of actions the agent can choose, $P(s' \mid s, a)$ is the transition function that gives the probability of reaching state s' after taking action a in state s, $r(s, a)$ is the immediate reward returned by the environment, and $\gamma \in [0, 1]$ is the discount factor that balances immediate and future rewards [14].

The return

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

is the discounted sum of all rewards collected from time step t onwards. The goal of control is to learn the optimal state–action value function

$$Q^*(s, a) = \mathbb{E}_{\pi^*}\big[G_t \mid s_t = s,\ a_t = a\big],$$

that is, the expected return if the agent starts in (s, a) and then behaves optimally. Q^* satisfies the Bellman expectation equation

$$Q^*(s, a) = \mathbb{E}_{s' \sim P(\cdot \mid s,a)}\big[\, r(s, a) \ + \ \gamma\, Q^*\big(s', \pi^*(s')\big)\big], \tag{1}$$

which states that the value of a state–action pair (s, a) equals the immediate reward $r(s, a)$ plus the discounted return obtained by following the optimal policy's greedy action in the successor state s'.

Function approximation and replay. In deep Q-learning the value function is represented by a neural network $Q_\theta(s, a)$ and trained with stochastic gradient descent. The loss minimised at every update is the squared temporal-difference (TD) error

$$\mathcal{L}_{\mathrm{TD}} = \mathbb{E}_{(s,a,r,s') \sim \mathcal{B}}\big[r + \gamma \max_{a'} Q_{\mathrm{tgt}}(s', a') - Q_\theta(s, a)\big]^2, \tag{2}$$

where samples are drawn uniformly from a replay buffer $\mathcal{B}$. The term Q_{tgt} denotes a target network, $Q_{\mathrm{tgt}}(s, a) = Q_{\theta^-}(s, a)$, whose parameters θ^- are a delayed copy of the online weights θ; every C steps the equality $\theta^- \leftarrow \theta$ is performed and the target is refreshed. Experience replay removes temporal correlations, and the slowly updated target network keeps the bootstrap term in (2) nearly stationary, both of which are essential for stable training [5].

Over-estimation bias. Because $\max_{a'} Q_{\mathrm{tgt}}$ selects the action with the largest noisy estimate, the TD target is optimistically biased. Double DQN mitigates this by decoupling action selection and evaluation **[15]**, yet the fundamental issue remains: the update treats the stochastic return G_t as a scalar—discarding information about its variance and tail behaviour. This motivates the distributional view adopted next.

2.2 Quantile-Regression DQN: learning a full return distribution

Why predict a distribution? In ordinary DQN the critic outputs a single number $Q_\theta(s, a)$, the expected discounted reward that follows action a in state s. In practice the return can vary widely from episode to episode, so reporting only its mean discards useful information. Distributional reinforcement learning therefore models the entire random variable $Z^\pi(s, a) = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$, not just its average. The resulting distributional Bellman operator is a strict γ-contraction, so repeated application converges to a unique fixed point [2].

Representing the distribution with quantiles. QR-DQN stores the return distribution as N trainable quantile values $\theta_i(s,a) \approx F_Z^{-1}(\tau_i)$ located at fixed probabilities $\tau_i = (i - \frac{1}{2})/N$. This parametrization keeps the support of the distribution fixed, eliminates the need for a softmax normalisation step, and reduces memory compared with the categorical C51 variant [3].

Learning rule. For a transition (s, a, r, s') we build targets

$$y_i = r + \gamma\, \theta'_i(s', a^\star), \qquad a^\star = \arg\max_{a'} \frac{1}{N} \sum_j \theta'_j(s', a'),$$

where the primed network $Q_{\theta'}$ is a target network, i.e., a frozen copy of the online parameters that is refreshed only periodically to stabilise the bootstrap targets. The parameters are updated by minimising the quantile-Huber loss [2]

$$\mathcal{L}_{\mathrm{QR}} = \frac{1}{N} \sum_{i=1}^{N} \rho_\kappa\big(y_i - \theta_i(s,a)\big), \qquad \rho_\kappa(u) = \begin{cases} \frac{1}{2}u^2, & |u| \le \kappa, \\ \kappa\big(|u| - \frac{1}{2}\kappa\big), & |u| > \kappa, \end{cases}$$

where κ is the Huber transition point between the quadratic and linear regimes.

Using the quantiles. During action selection the agent still needs only a single number—the mean of the quantiles, $Q(s,a) = \frac{1}{N}\sum_i \theta_i(s,a)$, so inference cost is identical to standard DQN. Because QR-DQN adds only a linear output layer, its computational overhead is negligible while still providing a much richer view of the agent's predictions.

3 Methodology

Contribution focus. The aim of this work is to show that a lightweight QR-DQN can learn efficiently when guided by three signals. Policy distillation supplies a low-cost starting policy, preference-based reward relabelling converts a handful of teacher-ranked clips into a dense shaping signal, and behaviour cloning provides self-improvement by briefly imitating the student's own highest-reward transitions. Their joint use is intended to shorten warm-up, mitigate sparse rewards, and steady value updates within a single, sample-efficient pipeline.

3.1 Policy distillation

After training a high-capacity teacher for a large number of steps, as shown in Figure 1A, we transfer its behaviour by minimising a temperature-scaled Kullback–Leibler divergence

$$\mathcal{L}_{\mathrm{KL}} = T^2\, \mathbb{E}_{s\sim\mathcal{D}_{\mathrm{obs}}} D_{\mathrm{KL}}\big(\mathrm{softmax}(Q_T(s)/T) \parallel \mathrm{softmax}(Q_S(s)/T)\big)$$

where T is the temperature parameter, π_T is the policy function of the teacher and π_S is the policy function of the student. The distillation dataset $\mathcal{D}_{\mathrm{obs}}$ contains M observations (98% on-policy teacher states; 1% random restarts; 1% ϵ-greedy exploration). The distillation step itself is depicted in Figure 1B, where the frozen teacher network provides soft targets that the smaller student tries to match.

3.2 Preference-based reward relabelling

Inspired by human-in-the-loop methods such as PEBBLE [4], we augment sparse environment rewards with a learned dense signal derived from pairwise trajectory rankings. At fixed intervals, the teacher samples two short clips, c^+ and c^-, from the student's replay and assigns a binary preference label $y \in \{0, 1\}$ indicating "which clip looks more successful". These labels train a small multilayer perceptron f_ϕ via the Bradley–Terry likelihood

$$\mathcal{L}_{\mathrm{BT}} = -\, y \log \sigma\big(f_\phi(c^+) - f_\phi(c^-)\big) - (1-y) \log \sigma\big(f_\phi(c^-) - f_\phi(c^+)\big),$$

where σ is the logistic function. Once updated, the network produces a per-state scalar $\hat{r}_t = f_\phi(s_t, a_t)$ that replaces the original environment reward for a fixed number of steps, thereby converting sparse, episodic feedback into a dense step-level shaping signal. Because the teacher's clip ranking is cheaper than computing a full return and uncertainty-aware querying limits how often preferences are requested, this mechanism supplies informative feedback without incurring heavy annotation overhead. Figure 1D shows how these labeled clips feed a tiny reward model that later supplies a dense score for every new transition.

3.3 Behaviour Cloning

After each environment step, the student receives the shaped reward estimate $\hat{r}_t = f_\phi(s_t, a_t)$ produced by the preference network introduced in Section 3.2. If this score is larger than a fixed threshold τ, the corresponding state–action pair (s_t, a_t) is copied into a dedicated first-in–first-out buffer $\mathcal{B}_{\mathrm{BC}}$. Because the teacher is not queried at this stage, $\mathcal{B}_{\mathrm{BC}}$ contains only transitions that the student itself has already executed with above-average return.

During learning, a mini-batch of size B is first sampled from the standard replay memory and used for the usual temporal-difference update. If at least B elements are available in $\mathcal{B}_{\mathrm{BC}}$, a second mini-batch of the same size is drawn from this buffer and optimised with the negative log-likelihood objective

$$\mathcal{L}_{\mathrm{BC}} = \lambda_{\mathrm{BC}} \frac{1}{B} \sum_{(s,a)\sim\mathcal{B}_{\mathrm{BC}}} \big[-\log \pi_S(a \mid s)\big],$$

where $\pi_S(\cdot \mid s)$ denotes the current student policy and λ_{BC} modulates the strength of the cloning signal. This auxiliary loss gently pulls the policy toward actions that have already yielded high rewards, accelerating credit assignment and reducing the risk of severe overestimation. As illustrated in Figure 1C, only those transitions whose shaped reward clears the threshold τ enter the cloning buffer.

3.4 Algorithm Design

To implement a teacher-guided reinforcement learning framework that combines offline and online training, we propose a multiphase algorithm, formally detailed in Algorithm 1. The pseudocode encapsulates the full training pipeline for a QR-DQN student policy, enhanced through distillation, preference-based reward modelling, and behaviour cloning. Each phase builds upon the last to ensure a robust and sample-efficient learning process.

Phase 0 begins by training a teacher policy π_T using standard QR-DQN for a fixed number of environment steps N_T. Once trained, the teacher is frozen and used as an expert policy to guide the subsequent student model.

Phase 1 performs policy distillation by constructing an observation bank $\mathcal{D}_{\mathrm{obs}}$ primarily from trajectories of the teacher. A student model π_S is trained to imitate the teacher's action-value distributions through a temperature-scaled KL divergence loss $\mathcal{L}_{KL}$ between the softmaxed teacher and student outputs. This step provides the student with a strong initialization that reflects expert behaviour.

Phase 2 introduces preference-based reward relabelling. Teacher trajectories are collected and categorized into positive or negative clips based on return thresholds θ^+ and θ^-. If necessary, additional negative samples are collected through random rollouts. These clips are then used to train a reward network f_ϕ using the Bradley–Terry loss, which encourages the network to assign higher scores to clips from successful trajectories. This learned reward replaces the environment reward during student training.

Phase 3 involves online reinforcement learning by the student, using both the learned reward function and behaviour cloning. The student explores the environment, and at each step, the environment reward is replaced with a relabeled reward $\hat{r}_t = f_\phi(s_t, a_t)$. Transitions are stored in a replay buffer $\mathcal{B}$, and those that exceed a reward threshold τ are also stored in a separate buffer $\mathcal{B}_{\mathrm{BC}}$ for BC. The student then alternates between standard QR-DQN TD updates and BC updates

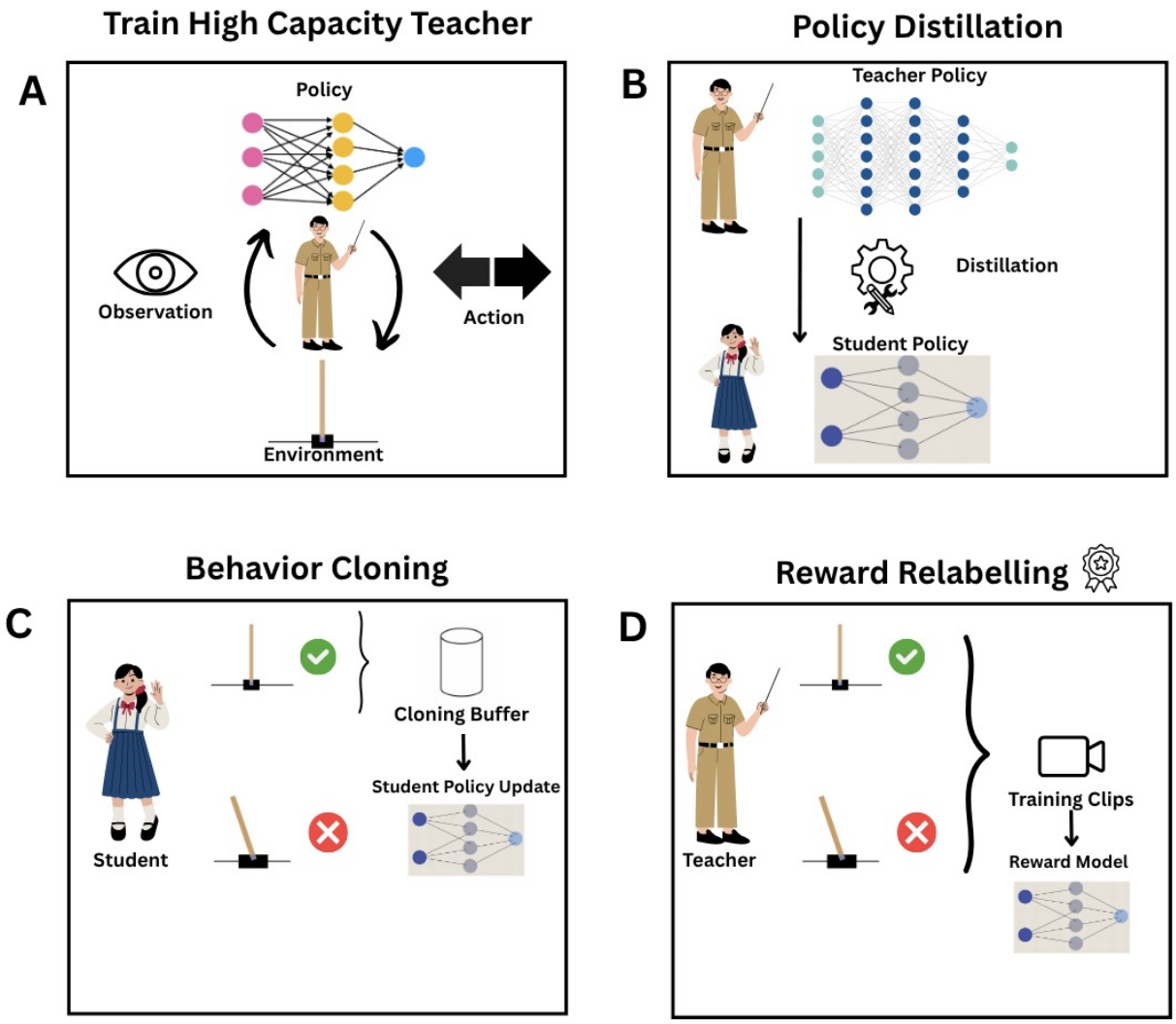

Figure 1: Teacher-guided learning pipeline. Panel A: teacher training; Panel B: policy distillation; Panel C: behaviour cloning; Panel D: preference-based reward relabelling.

(when available). Additionally, the reward model is periodically fine-tuned every F steps using the same preference dataset, enabling it to remain adaptive as the student encounters new state distributions.

This algorithm synergizes expert guidance, preference-informed reward shaping, and high-confidence imitation learning into a coherent training strategy. It ensures both alignment with expert behaviour and adaptability through continued self-improvement.

Algorithm 1: Phase 0 — Train High-Capacity QR-DQN Teacher

Input: Total teacher steps (e.g., 200,000)
Output: Trained and frozen teacher policy

1 Initialize a QR-DQN agent with large model size (e.g., 128–128 MLP, 50 quantiles)
2 Train the agent in the environment for the full number of steps
3 Freeze the final weights to define the teacher policy

Algorithm 2: Phase 1 — Distill Policy from Teacher to Student

Input: Frozen teacher, temperature T, number of epochs
Output: Initialized student policy

```
Collect a dataset of observations from the teacher's gameplay
Include some rare or random states for diversity
for each distillation epoch do
    Sample a batch of observations
    Get the teacher's action probabilities using softmax with temperature T
    Train the student model to match these probabilities
    Update the student's weights
```

Algorithm 3: Phase 2 — Learn Reward Model from Ranked Episodes

Input: A set of teacher episodes, return thresholds
Output: A trained preference-based reward model

```
Run many episodes using the teacher policy
Label episodes as 'good' if return is high, 'bad' if return is low
If there are too few bad ones, include random policy runs
Train a small reward model to prefer good episodes over bad ones
Use a ranking loss to update the model's weights
```

Algorithm 4: Phase 3 — Train Student with Reward Model and behaviour Cloning

Input: Reward model, total student steps
Output: Final student policy

```
Initialize empty replay and cloning buffers
for each training step do
    Let student take an action and observe next state and reward
    Replace the original reward with the learned one from the reward model
    Store the transition into the replay buffer
    If reward is high, also store the transition for cloning
    Sample a mini-batch from the replay buffer and update the student using QR-DQN
    Occasionally sample from the cloning buffer and apply a penalty when the student
      deviates from high-quality teacher actions
```

4 Experiments

We evaluate the teacher–student pipelines on the Cart-Pole control task (4.1), specify the teacher and student hyper-parameters (4.2), and analyse the impact of each guidance signal (4.3). All results are averaged over fifty random seeds and reported every 5,000 environment steps (to match the evaluation frequency used in plots and setup; see RQs a–d).

4.1 Cart-Pole Environment

Cart-Pole is a four-dimensional benchmark that is expressive enough to reflect sample efficiency yet simple enough to isolate algorithmic differences. The task features continuous state ($s = (x, \dot{x}, \theta, \dot{\theta})$, where x is the cart position, $\dot{x}$ the velocity, θ the pole angle, and $\dot{\theta}$ the angular velocity) and binary action $a \in \{0, 1\}$. At every time-step the agent applies a unit force left ($a = 0$) or right ($a = 1$). The environment returns $r = 1$ while the pole remains within $|x| \leq 2.4\,\text{m}$ and $|\theta| \leq 12°$, which defines a sparse reward setting. This sparse feedback makes the task suitable for testing preference-based reward relabelling (see RQb), as it allows us to examine whether learned dense rewards can improve exploration efficiency. Episodes end when these limits are breached or after 200 steps. As illustrated in Fig. 2, the objective is to keep the angle of the pole θ close to upright by oscillating the position

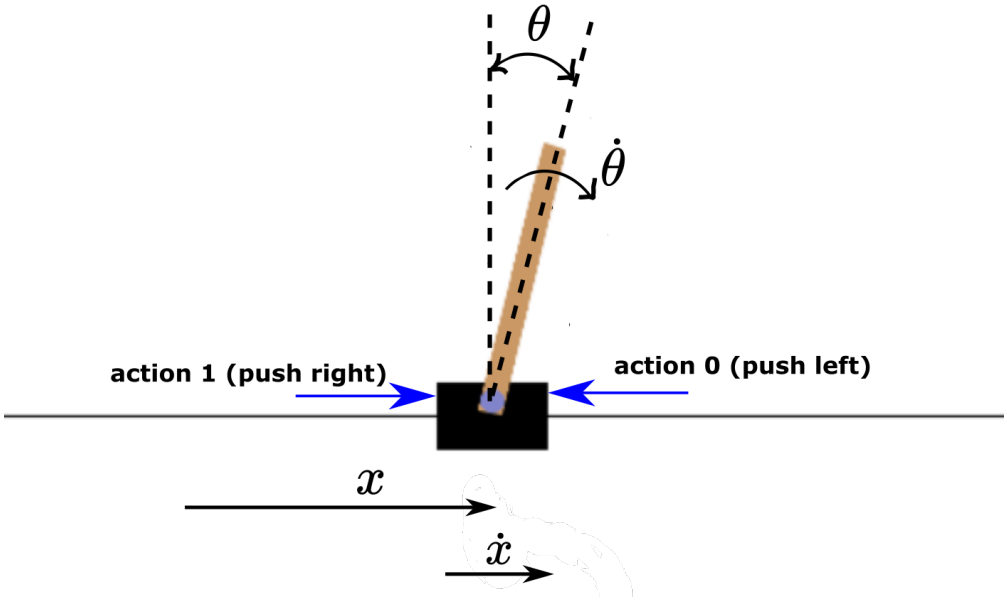

Figure 2: Schematic of the Cart-Pole dynamics. The agent chooses between two actions: push left (*action 0*) or push right (*action 1*). These forces translate the cart along the track (horizontal axis x), indirectly affecting the pole angle θ and angular velocity $\dot{\theta}$. Balancing consists of applying a sequence of forces that keep θ near the vertical dashed line while preventing the cart from leaving the track.

of the cart x within the limits of the track.

4.2 Model setup

Our methodology involves training a high-capacity QR-DQN as a teacher and evaluating four student models with increasing levels of supervision. These include: (1) a scratch-trained student, (2) a distilled student, (3) a student with preference-based rewards, and (4) a full pipeline including behaviour cloning. In addition, we evaluate combinations of these methods—e.g., distillation plus preference-based relabelling—to study their interactions (see RQd). All student models share the same QR-DQN architecture but operate with fewer parameters and constrained training budgets. A high-capacity QR-DQN with a 128-128 MLP body and 50 quantiles is trained for 200,000 environment steps and then frozen as the teacher policy π_T. An observation bank $\mathcal{D}_{\text{obs}}$ containing 120,000 states (98% on-policy teacher states; 1% random restarts; 1% ϵ-greedy exploration) feeds a twenty-epoch KL distillation phase at temperature $T = 0.1$. These settings were selected based on empirical tuning for stability and represent a trade-off between training cost and coverage of the state space.

From 400 teacher episodes, we extract clips whose returns exceed 450 or fall below 50; if fewer than 100 negative clips are available, random-policy episodes are added until the quota is met. A two-layer reward network f_ϕ with 32 hidden neurons is pre-trained for ten epochs on the Bradley–Terry pairwise ranking loss and then fine-tuned every 2,000 student steps. When active, this network replaces the original environment reward with $\hat{r}_t = f_\phi(s_t, a_t)$. Transitions with $\hat{r}_t > \tau$ populate the behaviour cloning buffer $\mathcal{B}_{\text{BC}}$; an additional negative log-likelihood loss weighted by $\lambda_{\text{BC}} = 0.1$ nudges the student policy π_S towards the teacher's preferred actions. The student itself uses a 2×32 MLP with 25 quantiles, the Adam optimiser at 2×10^{-4}, a replay buffer of 50,000 transitions, and target network updates every 500 steps.

Four variants are compared with identical 50,000-step training budgets: a scratch baseline with no teacher signal, pure distillation, distillation combined with preference-based rewards, and the full pipeline that additionally includes behaviour cloning. All experiment were repeated with fifty different random seeds, and performance is evaluated every 5,000 steps over ten independent episodes.

4.3 Effect of Teacher–Guided Distillation

Policy distillation transfers the softened action-value distribution of an expert QR-DQN teacher to a compact student by minimizing the KL divergence between their logits. The temperature hyper-parameter T controls the information bandwidth of this channel: small values sharpen the distribution and enforce near-hard imitation, whereas larger values smear probability mass over sub-optimal actions, promoting exploration at the cost of weaker guidance. To isolate this effect (RQa), we repeated the full training pipeline, holding every setting fixed except $T \in \{0.1, 0.5, 1.0, 2.0\}$ and adding a SCRATCH control with no teacher.

Teacher guidance remains unequivocally advantageous over the longer horizon: every distilled student overtakes SCRATCH well before the half-way mark and the gap widens thereafter. As summarised in Figure 3A, the sharpest targets ($T = 0.1$) yield the highest mean reward (200.9 ± 116.8), while intermediate softening ($T = 0.5$–1.0) trades a modest drop in peak score for noticeably tighter dispersion. Raising T further to 2.0 partially recovers the mean but not the variance, reflecting noisier gradients. All distilled variants, however, outpace the scratch baseline (114.7 ± 64.8), confirming that even a single-pass distillation provides a strong performance prior. The slightly higher standard deviations observed for the distilled models indicate that stronger, temperature-modulated gradients amplify inter-seed variability, yet this risk is outweighed by their substantial gains in sample efficiency.

Student variant	**Mean reward**	**Std. dev.**
Scratch (no teacher)	114.7	64.8
Distil, $T = 0.1$	**200.9**	**116.8**
Distil, $T = 0.5$	173.4	98.3
Distil, $T = 1.0$	179.5	106.0
Distil, $T = 2.0$	186.3	107.3

Table 1: Mean and standard deviation of final episode rewards for the scratch student versus distilled students at different temperatures T.

4.4 Effect of Self Improvement: Behaviour-Cloning

In our version of behaviour cloning (BC), the student learns from its own best moves. Besides the usual TD loss, we add a cross-entropy loss on the student's "good" transitions—the ones whose shaped reward falls in the top few percent of recent steps. This extra loss gently steers the policy toward actions that have already proved successful. The auxiliary weight λ_{BC} regulates the strength of this signal: small values leave the TD updates mostly intact, whereas large values can overwhelm the reinforcement gradient and introduce an early bias. To dissect this trade-off (RQc), we trained QR-DQN students from scratch with $\lambda_{\mathrm{BC}} \in \{0.1, 0.5, 1.0\}$ and a SCRATCH control ($\lambda_{\mathrm{BC}} = 0$). Figure 3B overlays the resulting learning curves (means over seeds).

Teacher-free guidance still proves valuable: $\lambda_{\mathrm{BC}} = 0.1$ more than doubles the SCRATCH return by the end of training and maintains a clear margin throughout, while moderate pressure ($\lambda_{\mathrm{BC}} = 0.5$) offers a milder, yet consistent boost. In contrast, pushing the weight to unity stalls progress: the final performance barely exceeds the baseline, implying that excessive imitation suppresses credit assignment. Figure 3B quantifies the outcome: the gentle auxiliary (211.0 ± 93.6) dominates, while the stronger BC both lowers the mean and inflates the variance, reflecting the louder optimisation. In general, a light BC regulariser strikes the best exploration–exploitation balance without sacrificing stability.

4.5 Effect of Preference-Based Reward Relabelling

Preference relabelling (PREF-RL) equips the student with a learned reward model that is trained, before any policy learning, on pairs of trajectory clips. Here, a clip is an entire CartPole episode—up

Student variant	Mean reward	Std. dev.
Scratch (no BC)	108.7	51.5
BC, λ=0.1	**211.0**	**93.6**
BC, λ=0.5	135.1	82.9
BC, λ=1.0	118.6	83.8

Table 2: Mean and standard deviation of final episode rewards for the scratch baseline and self-imitation variants with different cloning weights λ.

to 200 time-steps—stored as the sequence of observations and corresponding actions collected by the teacher. A set of M "positive" (>450 return) and n_{neg} "negative" (<50 return) episodes forms the dataset; the resulting network replaces the environment reward during training but the teacher is never consulted again. To quantify how much supervisory data are required (RQb), we ran three settings—$(M, n_{\text{neg}}) \in \{(100, 25), (200, 50), (400, 100)\}$—next to a Scratch baseline, keeping distillation and behaviour cloning disabled. Figure 3C plots the mean learning curves, with shaded bands showing ± 1 standard error of the mean (s.e.m.).

All preference-augmented agents surpass the unguided learner within the first 20,000 steps and converge to markedly higher plateaus. Figure 3C reports the pay-off after 50,000 interactions: even the smallest dataset (M=100) lifts the mean return from 114.6 to 186.6 without inflating the variance, while doubling the clip budget to $(200, 50)$ or $(400, 100)$ provides no further statistically meaningful gains. This saturation suggests that the reward function for CartPole can be induced reliably from only a few hundred teacher demonstrations, and that additional clips yield diminishing returns. In practice, therefore, Pref-RL offers a powerful yet data-efficient alternative to external reward engineering.

Student variant	Mean reward	Std. dev.
Scratch (no Pref)	114.6	59.7
Pref, M=100, n_{neg}=25	**186.6**	59.0
Pref, M=200, n_{neg}=50	184.9	**53.0**
Pref, M=400, n_{neg}=100	184.9	59.9

Table 3: Mean and standard deviation of final episode rewards for the scratch baseline and preference-relabelling students trained with three clip datasets of size (M, n_{neg}).

4.6 Complementarity of the Three Signals: Distillation, Preferences, and Behaviour Cloning

To assess how the three supervision channels interact (RQd) we trained eight QR-DQN variants obtained by toggling each signal on or off: the unguided Scratch baseline; the single-signal models Distil, Pref, and BC; the two-way hybrids Distil+Pref, Distil+BC, and Pref+BC; and the full tri-modal Distil+Pref+BC. Figure 3D presents the learning curves and final-return statistics.

The overall picture is hierarchical. Each single guidance channel already raises the final score well above Scratch (114.9 ± 60.8): distillation provides a strong prior (182.2 ± 95.7), preference relabelling adds dense feedback (184.8 ± 52.8), and self-imitation boosts credit assignment (200.9 ± 80.4). Combining distillation with another signal yields larger gains, though in different ways: Distil+Pref reaches its plateau fastest and ends at 244.1 ± 98.9, whereas Distil+BC climbs more gradually to 222.8 ± 99.1. Adding BC on top of Distil+Pref nudges the mean to 245.1 ± 95.2, but this increment over Distil+Pref is not significant. In that regime BC mainly reduces early-training variance and remains valuable when preference labels are scarce. Together the three techniques address distinct failure modes—cold start, sparse rewards, and unstable updates—and their combination delivers the most consistent performance (Table 4).

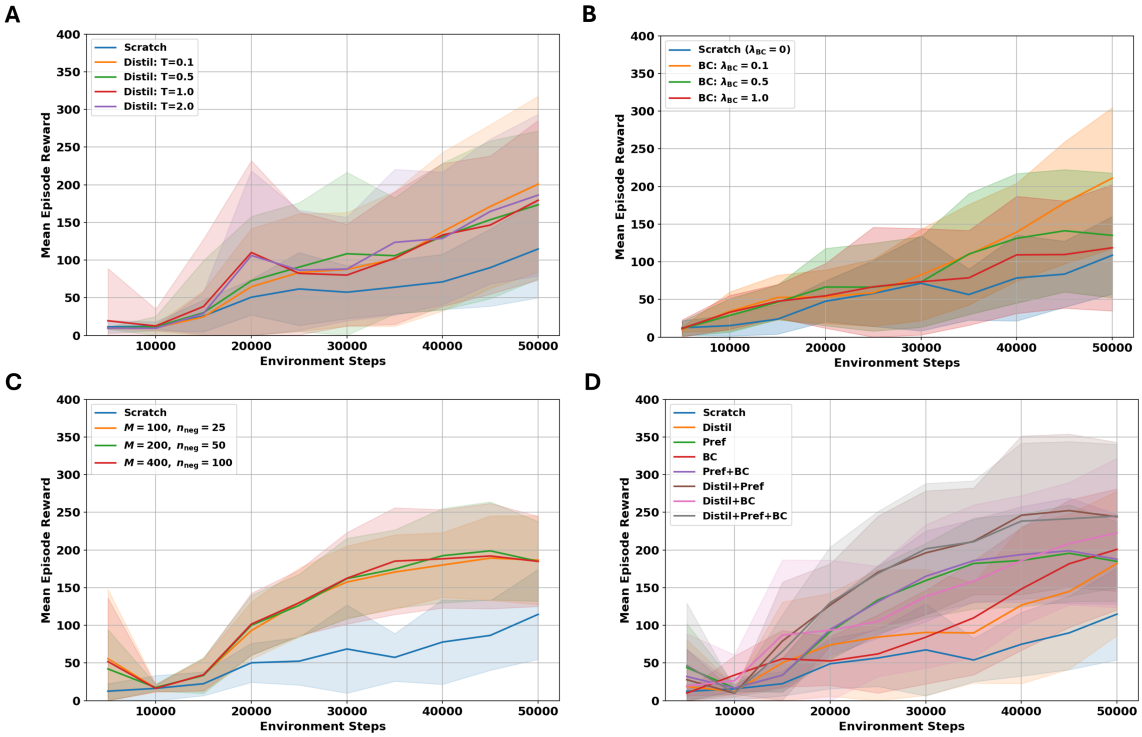

Figure 3: Learning curves for ablation experiments on CartPole. Each panel isolates the contribution of a single supervision channel: (A) policy distillation at various temperatures, (B) behaviour cloning with different penalty weights, (C) preference relabelling with increasing clip quantities, and (D) all combinations of the three methods. Results show that each component improves learning over the scratch baseline, and their combination yields the best performance.

5 Conclusions and Discussion

The thesis asked how much a small QR–DQN student can benefit from three teacher-derived signals while learning CartPole under a strict interaction budget. Four research questions (RQa–RQd) framed the study; each is answered below in its own paragraph.

RQa *Does policy distillation help an under-sized student?* Transferring the teacher's softened action distribution lifts the student out of its blank-slate start: after only a few thousand interactions it can already balance the pole for an entire episode, something the scratch agent achieves only much later in the run. This warm start requires just one pass over the teacher's logits, yet it halves the data budget needed to reach the final performance plateau, showing that distillation is the strongest single boost among the three guidance signals.

RQb *Can a very small ranked dataset replace CartPole's sparse reward?* A reward model trained on just 125 teacher-ranked clips converts the binary "alive / failed" signal into a dense, per-step gradient. That shaped reward lowers learning variance and raises the final plateau, demonstrating that even limited preference information can steer exploration more effectively than the environment alone.

RQc *When does Behaviour cloning help rather than hinder?* Self-imitation is beneficial only in moderation. A light cross-entropy weight highlights actions the student has already executed with high reward and thus speeds credit assignment, but larger weights dominate the RL gradient and arrest improvement. behaviour cloning should therefore be treated as a gentle regulariser, not the main learning signal.

Student variant	Mean reward	Std. dev.
Distil+Pref+BC	**245.1**	95.2
Distil+Pref	244.1	98.9
Distil+BC	222.8	99.1
BC	200.9	80.4
Pref+BC	187.7	59.5
Pref	184.8	52.8
Distil	182.2	95.7
Scratch (no aux.)	114.9	60.8

Table 4: Mean and standard deviation of final episode rewards for all eight combinations of the teacher-derived signals: distillation, preference relabelling, and behaviour cloning.

RQd *How do the three signals interact?* Each tackles a different bottleneck—cold start, sparse feedback and credit assignment—so their gains add rather than clash. The full tri-modal agent reaches the success threshold several times faster than a scratch learner and achieves the highest mean return, confirming that modest doses of all three signals outperform any single signal pushed to its limit.

Limitations and Future Work The main limitation is that all experiments were run on CartPole, a four-state, two-action, nearly deterministic domain. Success here does not guarantee success in environments that add (i) high-dimensional pixel inputs, (ii) continuous action spaces, or (iii) delayed, noisy rewards. Vision tasks will require a frozen convolutional encoder so the student and teacher share a feature space; the preference model may need longer clips and many more rankings to cope with visual variability; and behaviour-cloning thresholds that work for binary actions may either starve or flood a continuous-action buffer. Evaluating the same pipeline on Atari, DeepMind Control Suite, or MuJoCo locomotion would therefore be the most direct next step and would reveal whether the observed efficiency gains carry over when perception, precision control and credit assignment all become harder.

Closing statement. A single frozen expert, mined for policy, preference and self-imitation cues, reduces the student's data budget by roughly a factor of 2.5 while achieving high mean episode rewards. Teacher-guided, distributional RL therefore offers a practical path toward data-frugal learning in tasks where every interaction is costly.

References

[1] Paul F. Christiano, Jan Leike, Tom B. Brown, et al. Deep reinforcement learning from human preferences. In *Proceedings of the 31st Conference on Neural Information Processing Systems (NeurIPS)*, 2017.

[2] Marc G. Bellemare, Will Dabney, Rémi Munos. A distributional perspective on reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.

[3] Will Dabney, Georg Ostrovski, David Silver, et al. Distributional reinforcement learning with quantile regression. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence (AAAI)*, 2018.

[4] Kimin Lee, Yoonho Lee, Kibok Lee, et al. PEBBLE: Feedback-efficient interactive reinforcement learning via relabelling experience and unsupervised pre-training. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021.

[5] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

[6] David Silver, Julian Schrittwieser, Karen Simonyan, et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362:1140–1144, 2018.

[7] Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, et al. Solving Rubik's Cube with a robot hand. *arXiv:1910.07113*, 2019.

[8] Matteo Hessel, Joseph Modayil, Hado van Hasselt, et al. Rainbow: Combining improvements in deep reinforcement learning. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence (AAAI)*, 2018.

[9] Geoffrey Hinton, Oriol Vinyals, Jeff Dean. Distilling the knowledge in a neural network. *arXiv:1503.02531*, 2015.

[10] Andrei A. Rusu, Sergio Gomez Colmenarejo, Caglar Gulcehre, et al. Policy distillation. In *Proceedings of the 4th International Conference on Learning Representations (ICLR)*, 2016.

[11] Julian Ibarz, Ilya Sutskever, Paul Christiano, et al. Reward learning from human preferences and demonstrations in Atari. In *Proceedings of the 32nd Conference on Neural Information Processing Systems (NeurIPS)*, 2018.

[12] Stéphane Ross, Geoffrey Gordon, Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.

[13] Ashvin Nair, Abhishek Gupta, Vitchyr Pong, et al. Accelerating online reinforcement learning with offline datasets. *IEEE Robotics and Automation Letters (RA-L)*, 2020.

[14] Richard S. Sutton & Andrew G. Barto. *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[15] Hado P. van Hasselt, Arthur Guez, David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence (AAAI)*, pp. 2094–2100, 2016.